

AgentTelemetry: An Open-Source Toolkit for Fault Detection in LLM Agent Systems

Krishna Chaitanya Balusu
Independent Researcher
United States
krishnabkc15@gmail.com

Abstract

Autonomous AI agents built on large language models (LLMs) fail in ways that standard distributed tracing cannot diagnose: reasoning loops, circular delegation, guardrail bypass, and planning failures all manifest as opaque INTERNAL spans in OpenTelemetry (OTel). We present AGENTTELEMETRY, an open-source, pip-installable Python toolkit (Apache-2.0) that extends OTel with a nine-kind Domain-Specific Metamodel (DSM) for AI-agent telemetry—five of the kinds are absent from both the OpenTelemetry GenAI conventions and OpenInference—enabling typed detector implementations for 14 agent fault types in a deterministic fault-injection harness. The toolkit provides adapters for seven agent frameworks, four analysis modules, and a real-time circuit breaker.

In a controlled detector-correctness evaluation under deterministic fault injection (rate 1.0, mocked LLM clients) across 3,528 runs (14 faults—13 of which align to the MAST multi-agent failure taxonomy with mapping frozen pre-detector-implementation— $\times$ 6 telemetry conditions $\times$ 7 frameworks $\times$ 6 mocked foundation models) plus 252 fault-free controls, two AI-agent baselines (OTel GenAI, OpenInference) and a vanilla-OTel control all converge to fault detection rate (FDR) 0.429 under typed-only comparison (detectors must dispatch on a typed enum attribute; permissive substring-matching is discussed in §4). AGENTTELEMETRY’s DSM averages FDR 0.547 on six third-party adapters (range 0.500–0.571; per-adapter gain +0.07 to +0.14; FPR rises 0.000→0.083 from one detector on one CrewAI workload). The qualitative coverage delta is the headline: the DSM is the only metamodel of the four that can express the eight coordination/cognitive/policy faults at all (Table 3). On 112 SWE-bench Lite instances a circuit-breaker intervention improves patch rate by +11 pp (5-seed percentile bootstrap, approx. 95% CI [+3,+18]) over a no-intervention control with identical agent/model/tools/prompt/budget; 75% of failures (Wilson 95% CI [66%, 82%]) are reasoning loops. Median instrumentation overhead 11.7 μ s/span on Apple M4 Pro and 14.2 μ s on AWS c7i.xlarge.

Tool: `pip install agenttelemetry`

Code: <https://github.com/Krishnachaitanyakc/AgentTelemetry>

Artifact: Zenodo DOI 10.5281/zenodo.PLACEHOLDER

Video: <https://youtu.be/PLACEHOLDER>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

CCS Concepts

• **Software and its engineering** → **Software verification and validation**; *Software post-development issues*.

Keywords

AI agents, observability, OpenTelemetry, fault detection, LLM monitoring, distributed tracing

ACM Reference Format:

Krishna Chaitanya Balusu. 2026. AgentTelemetry: An Open-Source Toolkit for Fault Detection in LLM Agent Systems. In *Proceedings of IEEE/ACM 41st International Conference on Automated Software Engineering (ASE '26)*. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

LLM-based autonomous agents—systems that pursue goals through iterative reasoning, planning, tool use, and delegation—are entering production deployments in customer support, code review, and incident response [1]. Frameworks such as LangChain, CrewAI, AutoGen, and LlamaIndex have accelerated adoption, but a critical gap remains: *there is no standardized observability infrastructure for diagnosing agent failures*.

OpenTelemetry (OTel) is the de-facto standard for distributed tracing. Its GenAI semantic conventions define operation types for LLM calls, tool execution, retrieval, and agent lifecycle. However, five agent-orchestration phases—planning, reasoning, safety-check monitoring, inter-agent delegation, and memory management—have no typed representation in either GenAI or OpenInference, rendering them as opaque INTERNAL spans. This semantic gap impedes automated fault detection for failure modes such as reasoning loops, circular delegation, guardrail bypass, and planning failures, because no detector can dispatch on a span kind that the trace never carries.

AGENTTELEMETRY is an open-source Python toolkit for SE researchers and engineers building, evaluating, or operating LLM-agent systems who need portable trace data and automated fault triage across frameworks. Three contributions:

- (1) **A Domain-Specific Metamodel (DSM)** of nine agent-specific span kinds (five novel) accompanied by six OCL well-formedness invariants, derived from systematic analysis of seven production frameworks and validated by ablation.
- (2) **A modular analysis toolkit** comprising four analysis modules (anomaly detection, cost aggregation, decision attribution, lexical-overlap grounding screen for triage) and a real-time circuit breaker.
- (3) **Seven framework adapters** using three instrumentation strategies (callback handlers, monkey-patching, manual API), requiring as few as 3 lines of code to instrument an agent.

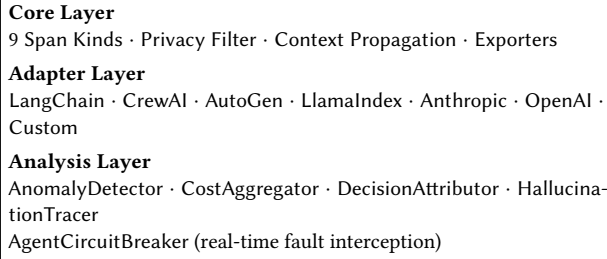


Figure 1: Three-layer architecture with OTel-native export.

Table 1: Nine agent-specific span kinds. ★ = novel (no OTel GenAI equivalent); ext. = extends OTel; ovlp. = overlaps with OTel.

Span Kind	Key Attributes	
AGENT	agent.name, .framework, .role	ext.
LLM_CALL	llm.model, .tokens, .cost	ovlp.
TOOL_CALL	tool.name, .input, .status	ovlp.
RETRIEVAL	retrieval.source, .doc_count	ovlp.
PLANNING ★	planning.strategy, .step_count	
REASONING ★	reasoning.chain	
GUARD_RAIL ★	guardrail.name, .result	
DELEGATION ★	delegation.src/tgt_agent	
MEMORY ★	memory.operation, .key	

The toolkit is pip-installable, OTel-native (all spans export via OTLP to any compatible backend), and adds 11.7 μ s of median in-process latency per span—negligible against typical LLM call latencies of 0.5–10 s.

2 Tool Architecture

AGENTTELEMETRY comprises three modules (Figure 1): **Core**, **Adapters**, and **Analysis**.

2.1 Core: Span Taxonomy

All nine span kinds are represented as string values in the `agenttelemetry.span.kind` attribute on standard OTel INTERNAL spans, ensuring compatibility with OTLP, Jaeger, and any OTel-compatible backend. Table 1 lists the kinds with their key attributes and novelty status relative to OTel GenAI.

The five novel kinds capture orchestration phases invisible to OTel GenAI: task decomposition (PLANNING), chain-of-thought (REASONING), safety-check outcomes (GUARD_RAIL), inter-agent handoff (DELEGATION), and state management (MEMORY). An ablation study confirms that every kind is necessary: removing any one makes at least one of 14 fault types undetectable.

A three-level privacy filter (NONE, METADATA_ONLY, FULL) controls attribute capture at the span level. The default (METADATA_ONLY) provides full diagnostic signal without exposing prompt or completion content.

The DSM is also accompanied by six OCL (Object Constraint Language) well-formedness invariants that the artifact ships as an executable `agenttelemetry.ocl` file (loadable into Eclipse

OCL or USE). For example, the *NoCircularDelegation* invariant requires that no two DELEGATION spans form a 2-cycle on (`delegation.src_agent`, `delegation.tgt_agent`); the others constrain LLM_CALL cost-attribute presence, GUARD_RAIL outcome enum, root-span kind, baggage propagation, and the kind/subclass discriminator.

2.2 Adapters: Three Instrumentation Strategies

Seven framework adapters implement three strategies. **Callback handlers** (LangChain, CrewAI, LlamaIndex; 903 LOC) register event listeners via framework extension APIs. **Monkey-patching** (Anthropic SDK, OpenAI SDK, AutoGen; 795 LOC) wraps key methods at runtime for SDKs without callback systems. **Manual API** (Custom; 315 LOC) provides context managers for explicit instrumentation. All framework imports are lazy (inside `_instrument()`), avoiding import-time dependencies.

2.3 Analysis Modules and Circuit Breaker

Four analysis modules operate on exported span data: **CostAggregator** sums token costs by model, agent, and trace; **DecisionAttributor** links each TOOL_CALL to its parent LLM_CALL and nearby REASONING spans; **AnomalyDetector** detects circular delegation, infinite retries, cost explosion, and context overflow; **HallucinationTracer** provides a lexical-overlap grounding screen suitable for trace-level triage—it is not a hallucination classifier and is intended to surface candidate traces for human review.

The **AgentCircuitBreaker** is an OTel SpanProcessor that intercepts fault patterns in real time. It dispatches on the typed span kind to check four policies: reasoning loops ($\geq N$ identical tool calls), cost explosion (cumulative cost threshold), context overflow (token growth rate), and delegation cycles (graph cycle detection). Each triggers a configurable callback (log, handler, or exception). An equivalent processor over vanilla OTel spans is possible but requires heuristic parsing of unstructured span names and free-form attributes; typed dispatch makes the policy code one line per fault.

Requirements. Python 3.10+; opentelemetry-sdk ≥ 1.27 ; the seven framework adapters lazily import their target SDK when `instrument()` is called, so installs do not pull in all seven by default. An `extras_require` group exists per framework (e.g. `pip install "agenttelemetry[langchain]"`). The artifact ships a Dockerfile and a `docker-compose.yml` that spins up an OTel collector and Jaeger backend reproducing all numbers in §4.

3 Usage

After `pip install agenttelemetry`, basic instrumentation is three lines (Listing 1); framework adapters auto-hook on a single adapter `.instrument()` call; spans export via OTLP to Jaeger, Grafana Tempo, Datadog, or any OTel-compatible backend; a JSON-Lines exporter supports offline analysis.

Listing 1: Three-line install + circuit-breaker.

```
from agenttelemetry import AgentTracer, AgentCircuitBreaker
tr = AgentTracer("research-agent")
tr.add_span_processor(AgentCircuitBreaker(max_reasoning_loops=3,
max_cost=0.50, on_trigger=lambda f: print(f))) # demo callback
with tr.start_agent_span("researcher"):
    with tr.start_llm_span("gpt-4o"): r = llm.chat(prompt)
    with tr.start_tool_span("search"): d = search_tool(query)
```

4 Evaluation

We evaluate AGENTTELEMETRY along four dimensions: fault detection effectiveness, span kind necessity, runtime overhead, and real-world applicability.

4.1 Fault Detection Rate

We define 14 fault types spanning all nine span kinds (e.g., reasoning loop, circular delegation, guardrail bypass, planning failure, cost explosion, hallucination); 13 of the 14 map onto modes of the MAST multi-agent failure taxonomy [1] (the 14th, COST_EXPLOSION, was added independently from public cost-runaway postmortems for autonomous-agent stacks). The taxonomy was finalized *before* we wrote the detectors, and the 9-kind DSM was finalized *before* the taxonomy was frozen; the artifact (§6) ships a `mast_mapping.md` with git-commit timestamps establishing this order (DSM $\rightarrow$ taxonomy $\rightarrow$ detector implementation). Each fault is injected at rate 1.0 into mock LLM clients. FDR here is operationalized as: under each metamodel, does a typed-enum-dispatch detector exist that fires on the injected fault? Baselines fail not because the underlying signal is absent from the trace’s free-form data but because no typed attribute exists for the detector to dispatch on; permissive substring-matching scoring is discussed below.

Across 3,528 fault-injection runs and 252 fault-free controls, the two AI-agent baselines (OTel GenAI, OpenInference) and a vanilla-OTel control all converge to FDR = 0.429 under typed-only comparison. All three detect the same six tool/LLM-attribute faults: `wrong_tool`, `tool_failure`, `timeout`, `infinite_loop`, `context_overflow`, `cost_explosion`. None expresses the eight coordination/cognitive/policy faults; in particular, OpenInference’s typed GUARDRAIL kind does not yield `guardrail_bypass` detection because the spec defines no constrained outcome attribute, and its typed AGENT kind does not yield `agent_misroute` detection because no typed inter-agent edge is captured. AGENTTELEMETRY’s DSM is the only metamodel of the four that *can* express those eight. It reaches a constructive upper bound of FDR = 1.000 on the conformance-complete reference adapter (`custom_agent`) and averages 0.547 across the six third-party adapters (Tables 2, 3). A permissive scoring rule allowing substring-matching on `span.name` would lift the baselines but at higher per-framework FPR; we use typed-enum-dispatch for the apples-to-apples metamodel comparison. False-positive rate is 0.000 for the three baselines and 0.083 for AGENTTELEMETRY, traced to the `infinite_loop` detector misfiring on a CrewAI workload that legitimately repeats a tool call across writer turns—a thresholding artifact, addressable with a state-aware OCL constraint (future work).

4.2 Ablation Study

For each of the nine span kinds, stripping all spans of that kind from the exported trace makes at least one fault undetectable. AGENT loses `agent_misroute`; LLM_CALL loses `cost_explosion`, `context_overflow`, `hallucination`, `infinite_loop`; TOOL_CALL loses `wrong_tool`, `tool_failure`, `infinite_loop`; RETRIEVAL loses `stale_retrieval`, `hallucination`; PLANNING/REASONING/GUARD_RAIL/MEMORY each lose their namesake fault; DELEGATION loses `circular_delegation` and

Table 2: Aggregate FDR / FPR by telemetry condition over 3,528 fault-injection runs and 252 fault-free controls. The headline DSM row is the third-party-adapter mean (n=6, the honest generalization). The conformance-complete reference adapter achieves 1.000 by construction (we built it to satisfy the OCL invariants); we report it in the ablation context (§4.2) rather than as a headline number.

Condition	FDR	FPR
No telemetry	0.000	0.000
Vanilla OTel	0.429 (6/14)	0.000
OTel GenAI (current spec)	0.429 (6/14)	0.000
OpenInference	0.429 (6/14)	0.000
DSM, third-party adapters (n=6 mean)	0.547	0.083

Table 3: Which fault types each metamodel can *express* (✓) under typed-only dispatch. The DSM uniquely covers the eight coordination/cognitive/policy faults below the rule. Per-adapter FDR for AgentTelemetry: `custom` 1.000 (by construction), `anthropic_sdk/autogen/crewai/openai_sdk` 0.571, `langchain/llamaindex` 0.500.

Fault	V	G	O	D	Fault	V	G	O	D
<code>wrong_tool</code>	✓	✓	✓	✓	<code>circular_deleg.</code>	–	–	–	✓
<code>tool_failure</code>	✓	✓	✓	✓	<code>agent_misroute</code>	–	–	–	✓
<code>timeout</code>	✓	✓	✓	✓	<code>planning_fail.</code>	–	–	–	✓
<code>infinite_loop</code>	✓	✓	✓	✓	<code>reasoning_loop</code>	–	–	–	✓
<code>context_overflow</code>	✓	✓	✓	✓	<code>guardrail_byp.</code>	–	–	–	✓
<code>cost_explosion</code>	✓	✓	✓	✓	<code>stale_retr.</code>	–	–	–	✓
					<code>memory_corr.</code>	–	–	–	✓
					<code>hallucination</code>	–	–	–	✓

Coverage: V=OTel 6/14, GenAI 6/14, OpenInference 6/14, DSM 14/14. V=Vanilla OTel, G=GenAI, O=OpenInference, D=DSM.

`agent_misroute`. No kind can be removed without losing at least one detection capability.

4.3 SWE-bench Case Study

On 112 SWE-bench Lite instances (37% of the benchmark, all 12 repositories), we instrument a ReAct coding agent (GPT-4o-mini, 5 tools, 8 iterations max, \$2.53 total API cost). The agent produces 3,060 spans across all nine kinds.

AGENTTELEMETRY characterizes reasoning loops as the dominant failure mode: 75% of 84 failed instances (Wilson 95% CI [66%, 82%]) repeatedly called the same tool with similar queries without converging. We define a reasoning loop as ≥ 3 TOOL_CALL spans whose `tool.name` and a 3-token-shingle Jaccard similarity ≥ 0.7 over the input arguments are identical; manual audit of 20 randomly-selected loop-flagged traces by the author gave 18 true loops (precision 0.90, single-author audit; inter-rater reliability not measured—the 75% reasoning-loop attribution inherits this measurement-validity gap; two false positives were legitimate refinement of an edit-file argument). This pattern is invisible to vanilla OTel, which cannot distinguish a reasoning step from any other INTERNAL span.

A telemetry-guided intervention—the circuit breaker injects a strategy-change prompt (verbatim: “*You have repeated the same tool call three times without progress. Stop, reconsider the goal,*

Table 4: Comparison with existing tools. ✓ = supported; ○ = partial; – = absent. Span kinds = typed agent-orchestration kinds beyond LLM/tool/retrieval. Privacy = declarative per-span attribute-capture levels with default redaction. Mng. SaaS = managed service with SLA. AgentTelemetry is research-grade software; LangSmith and Datadog provide commercial support and substantially larger production user bases.

Tool	Multi-fw	Span kinds	Multi-agent	Privacy	Fault det.	Open-src	OTel-native	Mng. SaaS
LangSmith [3]	–	○	–	–	–	–	–	✓
Langfuse [4]	○	–	–	–	–	✓	✓	✓
OpenLLMetry [5]	✓	–	–	–	–	✓	○	✓
Datadog LLM [6]	✓	–	–	–	–	✓	○	✓
OTel GenAI [7]	✓	○	○	–	–	✓	✓	–
OpenInference [8]	✓	○	○	–	–	✓	✓	✓
AGDebugger [2]	–	○	✓	–	○	✓	–	–
AgentTelemetry	✓	✓	○	✓	✓	✓	✓	–

and try a different tool or argument shape”) after a flagged loop—improves the patch rate by +11 percentage points (5-seed percentile bootstrap of per-seed differences, approximate 95% CI [+3, +18]; baseline 25.0% (SD across 5 seeds: 3.4%), intervention 36.0% (SD: 3.1%); per-seed pairs reported in the artifact). Each seed runs all 112 instances with identical agent configuration (GPT-4o-mini, 5 tools, 8-iteration max, identical system prompt, fixed token budget); the only delta is whether the circuit breaker’s on_trigger callback is enabled. Per-seed logs and the exact prompt are in experiments/swebench_closed_loop.py. A sham-prompt arm (non-loop-related reflection prompt at the same trigger points) is in the artifact roadmap and will isolate the detection contribution from the reflection-prompt contribution.

4.4 Overhead

Measured over 90,000 iterations (10,000 per kind) on two reference machines: an Apple M4 Pro laptop and a 4 vCPU AWS c7i.xlarge cloud VM. Median span creation is 11.7 μs (M4) / 14.2 μs (c7i); p99 28.2 / 35.1 μs; memory ~3.1 KB per span; sustained throughput 58–66 k / 42–48 k spans/s. Given 1–10 spans/s and LLM latencies of 0.5–10 s, the per-span overhead adds <0.006% to end-to-end execution.

5 Related Work

Table 4 positions AGENTTELEMETRY against representative tools across seven dimensions critical to production agent observability.

Failure taxonomies. The MAST taxonomy of Cemri et al. [1] identifies 14 multi-agent failure modes from 200+ annotated traces; 13/14 of our fault types map to MAST modes (artifact: mast_mapping.md). These taxonomies characterize *what* goes wrong but provide no runtime detection infrastructure.

Debugging tools. AGDebugger [2] offers interactive visual debugging for multi-agent conversations. It operates on framework-specific logs; AGENTTELEMETRY provides standardized, OTel-native trace data that such tools could consume.

LLM observability platforms. LangSmith [3] provides deep LangChain integration but no portability to other frameworks. Langfuse [4] and OpenLLMetry [5] provide OTel-compatible tracing but define no agent-specific span kinds—a planning step and a summarization call are indistinguishable. Datadog LLM Observability [6] adds LLM metrics to a commercial stack but cannot detect agent orchestration faults.

Industry typed-span conventions. The OTel GenAI conventions [7] define typed operations covering LLM calls, retrieval, tools, agent creation, and agent invocation. OpenInference [8] defines ten typed span kinds including AGENT and GUARDRAIL. Both lift agent and tool to first-class types but neither exposes typed sub-operations for delegation topology, planning, reasoning, or memory, nor a constrained guardrail outcome. AGENTTELEMETRY is a strict typed-superset and coexists on the same OTLP export pipeline.

6 Conclusion

AGENTTELEMETRY extends OpenTelemetry with a nine-kind DSM, six OCL invariants, seven framework adapters, four analysis modules, and a real-time circuit breaker. Under typed-only comparison the DSM averages FDR 0.547 on third-party adapters (vs. 0.429 for vanilla OTel / OTel GenAI / OpenInference) and reaches 1.000 on a conformance-complete reference adapter; on SWE-bench Lite a circuit-breaker intervention improves patch rate by +11 pp (approx. 95% CI [+3,+18], 5 seeds). Median per-span overhead is <15 μs on commodity x86 cloud.

Limitations. Mocked LLMs with rate-1 fault injection (not real production faults); single-agent SWE-bench (5 seeds, lower bound +3pp); no sham-prompt control isolating detection from prompt-injection; single-author loop-detector audit (n=20); third-party adapter coverage uneven by framework, so the third-party gain reflects today’s adapter realization, not the metamodel ceiling; FPR gap (0.083 vs. 0.000) localizes to one CrewAI thresholding artifact, not a metamodel limitation.

The toolkit is on PyPI (pip install agenttelemetry, Apache-2.0); code at <https://github.com/Krishnachaitanyakc/AgentTelemetry>; artifact at Zenodo DOI 10.5281/zenodo.PLACEHOLDER. Future work: state-aware OCL to remove the CrewAI false positive; sham-prompt control on the SWE-bench intervention; permissive-baseline study; adaptive sampling for long-running agents.

References

- [1] M. Cemri, M. Z. Pan, S. Yang, et al. Why Do Multi-Agent LLM Systems Fail? *arXiv:2503.13657*, 2025.
- [2] W. Epperson, G. Bansal, V. Dibia, et al. Interactive Debugging and Steering of Multi-Agent AI Systems. In *Proc. CHI*, 2025.
- [3] LangChain. LangSmith: LLM Application Observability Platform. <https://smith.langchain.com>, 2024.
- [4] Langfuse. Open-Source LLM Observability. <https://langfuse.com>, 2024.
- [5] Traceloop. OpenLLMetry: OpenTelemetry-based observability for LLM applications. <https://github.com/traceloop/openllmetry>, 2024.
- [6] Datadog. LLM Observability. <https://www.datadoghq.com/product/llm-observability/>, 2024.
- [7] OpenTelemetry Contributors. GenAI Semantic Conventions. <https://opentelemetry.io/docs/specs/semconv/gen-ai/>, 2024.
- [8] Arize AI. OpenInference: Semantic Conventions for AI System Observability. <https://github.com/Arize-ai/openinference>, 2024.